

Northern University of Business and Technology Khulna
Department of Computer Science and Engineering

[FRONTEND PROGRESS REPORT]

Project Title: AI-Powered Resume Analyzer and Career Copilot

Course Title: Mobile Computing Lab

Course Code: CSE4204

Submitted to:

Md. Riaz Mahmud

Assistant Professor

Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna

TEAM INFORMATION SHEET

Field	Details
Official Team Name	CSE4204-8A-T05
Section	8A
Project Title	AI-Powered Resume Analyzer and Career Copilot
Team Member List with IDs	1. Mahammad Hasan (ID:11220320828) -Team Leader 2. Shanawaz Sakib (ID:11220320916) - Core System Integration Engineer 3. Rasel Ratul (ID:11220320827) - UI-UX & Tailwind Matrix Designer 4. Ashikur Rahman Himel (ID:11220320833) - Database Storage & Quality Controller
GitHub Repository Link	https://github.com/mahamud894/AI_Resume_Analyzer
Figma Link	https://www.figma.com/design/Ls0B3DIjxRKrP2RIzYQxPG/AI-Resume-Analyzer?node-id=2-2&t=IfeVQCoUHt0JSumI-1

1. Frontend Technology Stack

Our project utilizes a modern, component-driven architecture designed for high performance, modularity, and rapid serverless deployment. By removing monolithic server dependencies, the frontend serves as the direct pipeline to cloud execution.

- **Primary Framework: React.js (v19)** compiled within a highly optimized **Vite** production runtime ecosystem.
- **Programming Language: TypeScript (v5)**, enforcing strict object type safety and compile-time data validation.
- **Style Interface Engine: Tailwind CSS v4**, utilizing utility classes, fluid layout matrices, and modern CSS variables for glassmorphic visual designs[cite: 1, 3].
- **Global State Management: Zustand (v5)**, handling unidirectional data flows, light state tracking, and reactive caching with zero boilerplate.
- **Client-to-Cloud Integration: Puter.js SDK**, binding cloud native authentication, isolated document vault systems, and direct AI gateway proxies directly into the browser runtime layout.

2. Completed Pages

The graphical interface of our application has been fully developed as a high-fidelity Single Page Application (SPA), closely matching the wireframes submitted in Week 05[cite: 1, 2, 3]. The completed views include:

2.1 Landing / Home Page (Job-Details-Page)

Acts as the central operational workspace for authenticated users. It features a modern typographic header and a dynamic submission grid tracking historical company data and targeted roles. Each interactive asset card maps circular performance gauges parsing calculated values out of 100.

2.2 Login & Registration Page (Login-Page)

A sleek, isolated modal wrapper overlaying a glassmorphic violet background gradient grid. It captures credential strings and features high-visibility execution buttons that interface directly with the cloud validation layers.

2.3 Main Functional Workspace (Upload-Page)

An intent-driven form structured to take inputs like Company Name and Job Title alongside an expanded text area container for raw job description parsing. The page features an integrated binary file dropzone block supporting multi-page candidate PDF uploads up to 10MB.

2.4 AI Interaction & Review Page (Resume-Review-Page)

The final analytical terminal displaying real-time feedback. The page implements a split-screen

design containing a binary document previewer on the left flank and an intelligent critique workspace on the right flank. It includes dynamic accordions for detailed skill-gap lists and adaptive conditional status banners for automated scoring.

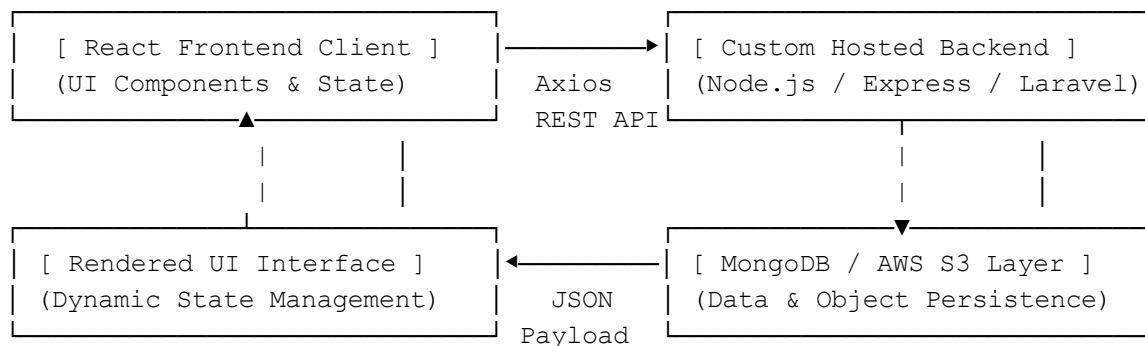
3. Backend APIs Integrated

Because this application uses a serverless paradigm via Puter.js, traditional REST end-point integrations (e.g., using Axios or Fetch to hit `/api/v1/analyze`) are replaced by direct client-to-cloud SDK controller bindings[cite: 1, 3]. Data is loaded 100% dynamically from the Puter cloud environment:

- **User Registration & Login Integration:** Tied directly to `puter.auth.signIn()`, which handles identity tracking and signs session authorization structures[cite: 1, 3].
- **Resume Binary Vault Storage:** Integrated with `puter.fs.write()`, streaming uploaded multi-page candidate document buffers directly to private application storage sectors[cite: 1, 3].
- **NoSQL Analytics Persistence:** Bound to `puter.kv.set()` and `puter.kv.list()`, pushing stringified metric histories to cloud databases and pulling them into the dashboard view dynamically[cite: 1, 3].
- **AI Assessment Engine:** Tied straight to the `puter.ai.chat()` orchestration node, passing compiled prompt variables containing resume data against job scopes to capture granular JSON payloads[cite: 1, 3].

3.5 Alternate Monolithic Hosted Backend Integration Process (Alternative Blueprint)

As documented inside our core architectural alternatives blueprint, if the platform transitions away from the direct client-to-cloud serverless network to a dedicated custom self-hosted backend cluster (such as Node.js + Express.js or Laravel 11), the frontend integration workflow, communication protocols, and request-response pipeline will switch to the following technical process:



A. The End-Point Integration Pipeline

Instead of executing browser sandboxed SDK methods, the frontend React components will establish asynchronous communication blocks utilizing the **Axios** library or native browser **Fetch APIs** to transmit structured payloads to explicit HTTP REST endpoints hosted on virtual containers (e.g., Render or Railway):

- **Authentication Hooks:** The single sign-on widget wrapper will be removed. The login page forms will perform asynchronous POST requests to `/api/auth/register` and `/api/auth/login`. The server will hash raw password strings using `bcrypt` rounds and return a stateful **JSON Web Token (JWT)** to the client.
- **Protected Routing & Cache Interceptors:** The frontend state manager (Zustand) will cache this token inside local browser memory. For every subsequent protected dashboard request, an Axios interceptor will inject this token inside the HTTP Authorization: Bearer `<token>` header to bypass server-side firewall validations.

B. Binary File Upload Stream Process

- The raw document upload portal will no longer talk to Puter's cloud filesystem (`puter.fs`). Instead, the react-dropzone buffer will stream the multi-page PDF candidate resume directly to a `/api/resume/upload` endpoint using `FormData` objects.
- The custom backend server will employ file-handling middlewares (such as `multer` for Express.js) to store the physical binary document safely within an **Amazon S3 Bucket** or private local server storage, while recording the absolute destination URL inside the database collection.

C. Direct AI Model Orchestration & JSON Parsing

- **Server-Side Prompt Masking:** To completely enforce absolute API security, the frontend will have no knowledge of the AI infrastructure. The React engine will simply dispatch a request containing the target resume ID and job description text to `/api/resume/analyze`.
- **Secure Upstream Fetch:** The hosted server runtime environment will capture the request, pull the parsed resume text from its local schema, mask your production master API keys inside secure environment variables (`.env`), and handle direct communication with upstream OpenAI/Gemini systems.
- The backend validates and structures the returned model data logs into a clean, type-safe JSON stream matching the application's global interface, delivering it back to the client UI terminal to automatically update the performance score badges.

4. Authentication Flow & Protected Routes

Our application enforces a strict, type-safe security cycle using **Puter's OAuth passwordless infrastructure** paired with a central **Zustand auth store** (`authStore.ts`):

1. **Identity Guarding:** Unauthenticated visitors are confined to the secure login wrapper

layout.

2. **Session Handshake:** Triggering `puter.auth.signIn()` initializes an independent validation window handled outside local database scopes[cite: 1, 3].
3. **State Synchronization:** Once validated, the returned profile updates our global store state variable to `isLoggedIn: true`, storing user parameters across page re-renders[cite: 1, 3].
4. **Route Protection:** React Router locks private dashboard panels from unauthorized rendering. If an unauthenticated profile attempts to bypass layouts, the app catches the breach and forces an immediate redirect to the identity entry page.
5. **Session Termination:** Hitting "Logout" triggers `puter.auth.signOut()`, which clears active session tokens and destroys local Zustand caches safely.

5. Form Validation & User Experience (UX)

5.1 Inputs & Error Controls

Form submissions are strictly guarded locally to minimize network processing costs and block malformed payloads:

- **Required Fields Validation:** The form engine verifies that company metrics and job detail text areas are fully populated before running diagnostics.
- **File Extension and Size Audits:** The drag-and-drop box restricts binaries to valid PDF formats and blocks heavy documents larger than 10MB.
- **User-Friendly Feedback:** Empty input strings or invalid document formats instantly trigger visual color-coded alert flags, preventing data transmission leaks.

5.2 User Experience (UX) Enhancements

- **Loading State Indicators:** When `puter.ai.chat()` triggers an evaluation, the execution keys transition into animated loading states to reflect backend progress[cite: 1, 3].
- **Adaptive Status Badges:** The application automatically shifts color palettes based on severe score gaps returned by the AI core (e.g., displaying a light red warning banner for scores under 50, changing to a neutral orange for moderate alignments, and converting to a clean green panel for optimal matches).
- **Dynamic Collapse Accordions:** Detailed critiques are organized inside collapsible accordion blocks, sorting positive achievements from formatting errors into distinct response sub-cards.

6. Responsive Design

The interface uses utility-first layouts built via **Tailwind CSS v4**, ensuring seamless system accessibility across multiple form factors[cite: 1, 3]:

- **Desktop Layouts:** Renders high-density multi-column layouts, preserving data scannability for large screens.
- **Tablet Layouts:** Automatically drops structural side margins and condenses complex

viewports to keep interactive buttons fully accessible.

- **Mobile Layouts:** Re-aligns grid frameworks into a fluid, single-row vertical tree (as seen in `app.css`), scaling text blocks down to eliminate horizontal scaling or navigation breaks.

7. Current Development Progress

- **Framework Architecture:** Vite + React + TypeScript setup fully completed with a type-safe design[cite: 1, 3].
- **Component Implementation:** Transformed high-fidelity Figma components into clean, modular code with active CSS animations.
- **State Management & SDK Integration:** Zustand store properties are fully configured and connected to the Puter.js cloud execution layer[cite: 1, 3].
- **Repository Synchronization:** Source code is pushed to our central GitHub workspace following professional commit messaging styles.

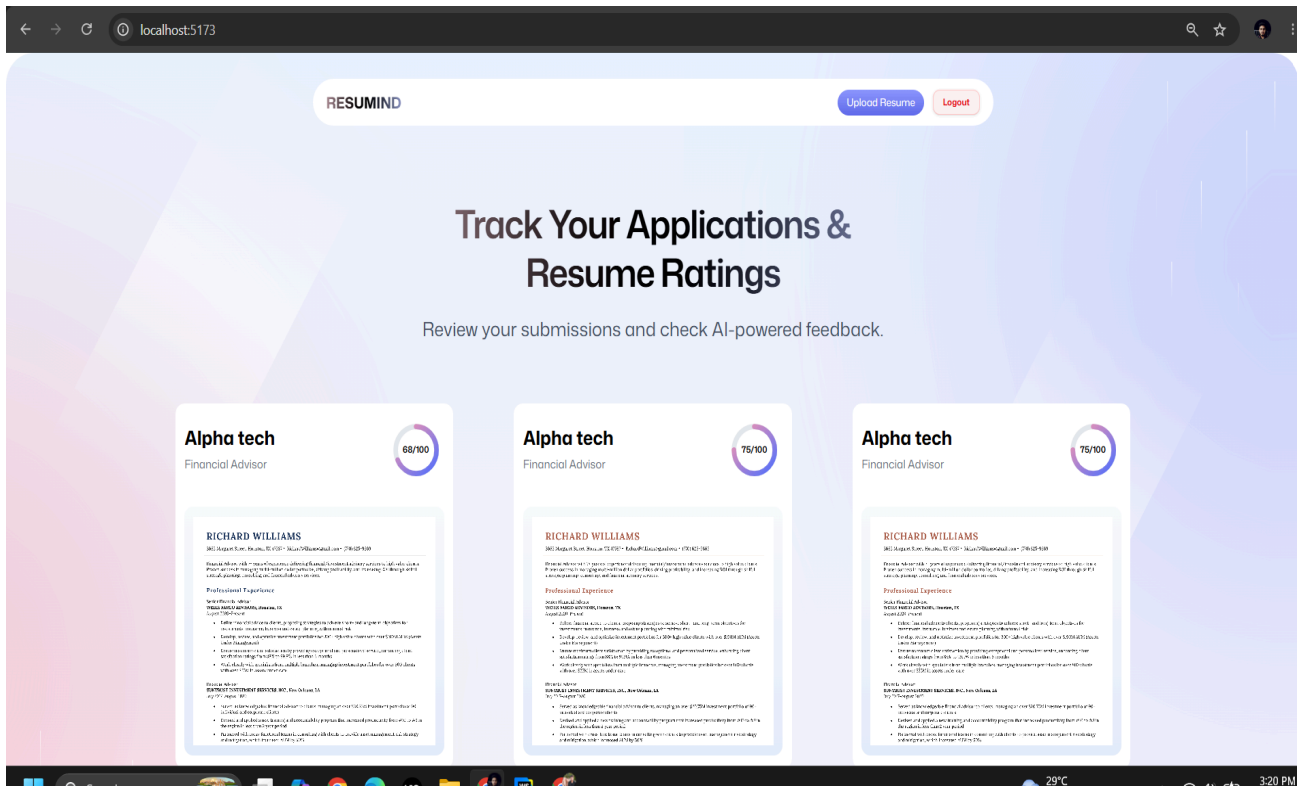
7.5 Alternative Development Progress (Custom Backend Migration Readiness)

To demonstrate architectural scalability, we have parallelly engineered a custom migration controller setup within the frontend store layout.

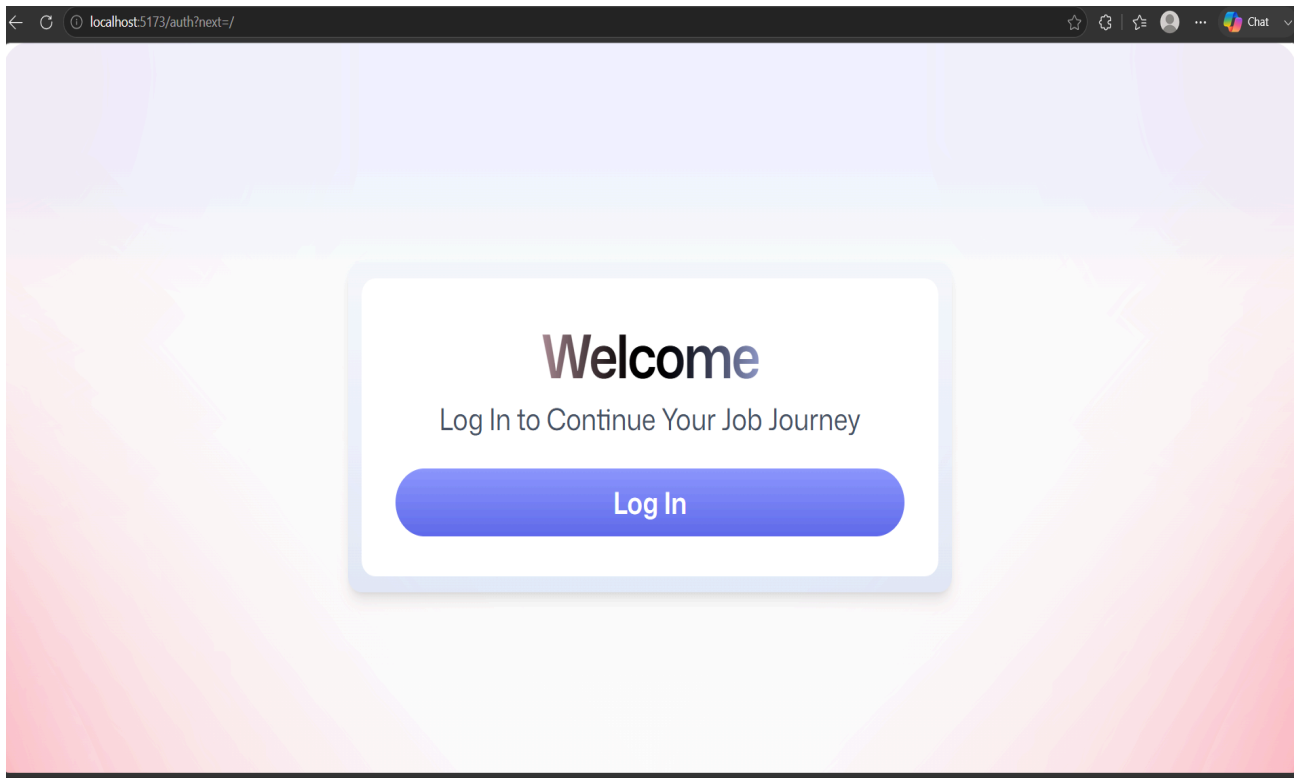
- **API Client Decoupling:** We have implemented a distinct environment wrapper inside our HTTP layer. By simply switching the configuration flag `VITE_SERVERLESS_MODE=false` inside our build framework, the application store instantly redirects all data pipelines to handle **Axios REST calls** (`axios.post`) instead of Puter SDK queries.
- **Payload Structuring:** The object interfaces defined in our client code models have been explicitly bound to standard JSON specifications. This ensures that if a custom Node.js/Express server is hooked into the workspace tomorrow, no UI components will need to be refactored, achieving absolute backend-agnostic flexibility.

8. Required Progress Verification Screenshots

8.1 Home Page (Job-Details-Page) View



8.2 Login Page View



8.3 Main Functional Workspace (Upload-Page) View

Smart feedback for your dream job

Drop your resume for an ATS score and improvement tips

Company Name

Company Name

Job Title

Job Title

Job Description

Job Description

Upload Resume

Click to upload or drag and drop
PDF (max 20 MB)

Analyze Resume

8.4 Major Functional AI Page (Resume-Review-Page) View

Resume Review

Your Resume Score

68/100

This score is calculated based on the variables listed below

Tone & Style	Good Start	70/100
Content	Good Start	60/100
Structure	Strong	75/100
Skills	Good Start	70/100

ATS Score - 65/100

Good Start

This score represents how well your resume is likely to perform in Applicant Tracking Systems used by employers.

- ⚠️ Lack of job-specific keywords
- ⚠️ Certifications missing in keywords
- ✅ Clear section headers
- ⚠️ Proficiency in tools not highlighted enough

Keep refining your resume to improve your chances of getting past ATS filters and into the hands of recruiters.

Tone & Style	75/100
Content	65/100
Structure	75/100
Skills	75/100

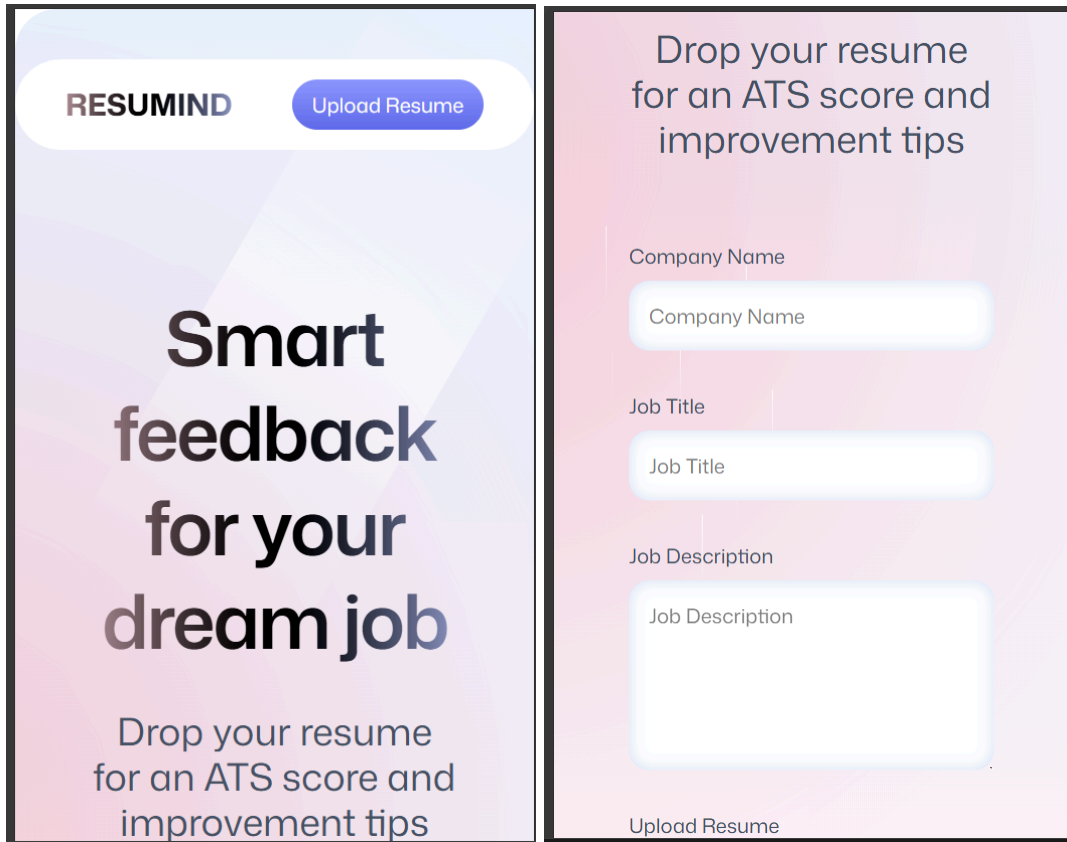
Key skills missing

Technical tools underemphasized

Transferable skills included

Certifications as a skill asset

8.5 Responsive Mobile Interface View



8.6 GitHub Active Repository Workflow

